DATABASE DESIGN & IMPLEMENTATION

ELEKTROMART – ELECTRONICS ONLINE STORE

TECHNICAL TEST TASK 1 – DATA ENGINEER INTERN

Disusun oleh : Marisa Muharomah

BUSINESS UNDERSTANDING

ElektroMart - Electronics Online Store

Business Overview

ElektroMart merupakan sebuah toko elektronik online yang menyediakan berbagai produk elektronik dan perlengkapan pendukung. Pelanggan dapat memilih produk, melakukan pemesanan, menyelesaikan pembayaran, dan menerima produk melalui layanan pengiriman.

Dalam menjalankan operasionalnya, ElektroMart membutuhkan sistem database yang dapat mengelola informasi pelanggan, produk, transaksi, pembayaran, dan pengiriman secara terstruktur.

BUSINESS NEEDS

01 Mengelola informasi pelanggan dan produk secara terstruktur.

02 Mencatat transaksi pemesanan dan detail produk yang dibeli.

03 Memantau status pembayaran setiap pesanan.

04 Memantau proses dan status pengiriman barang.

05 Menghasilkan informasi penjualan untuk mendukung pengambilan keputusan bisnis.

PROJECT OVERVIEW

Project ini bertujuan merancang dan mengimplementasikan database relasional ElektroMart menggunakan Oracle Database untuk mengelola data pelanggan, produk, pesanan, pembayaran, dan pengiriman secara terstruktur.

Tujuan Project

- Merancang dan mengimplementasikan struktur database relasional.
- Mengelola data bisnis dan transaksi.
- Membangun hubungan antar tabel menggunakan Primary Key (PK) dan Foreign Key (FK).
- Melakukan pengambilan dan manipulasi data menggunakan SQL.
- Menghasilkan informasi penjualan melalui analisis SQL.

DATA MODELING

Database ElektroMart menggunakan Relational Data Model, dengan data dikelompokkan ke dalam tabel-tabel yang saling berhubungan berdasarkan kebutuhan bisnis.

01

Entity & Attribute

Objek dalam database beserta informasi yang dimilikinya.

02

Primary Key (PK)

Identitas unik untuk setiap record.

03

Foreign Key (FK)

Penghubung antar tabel.

04

Relationship

Hubungan antar entity dalam database.

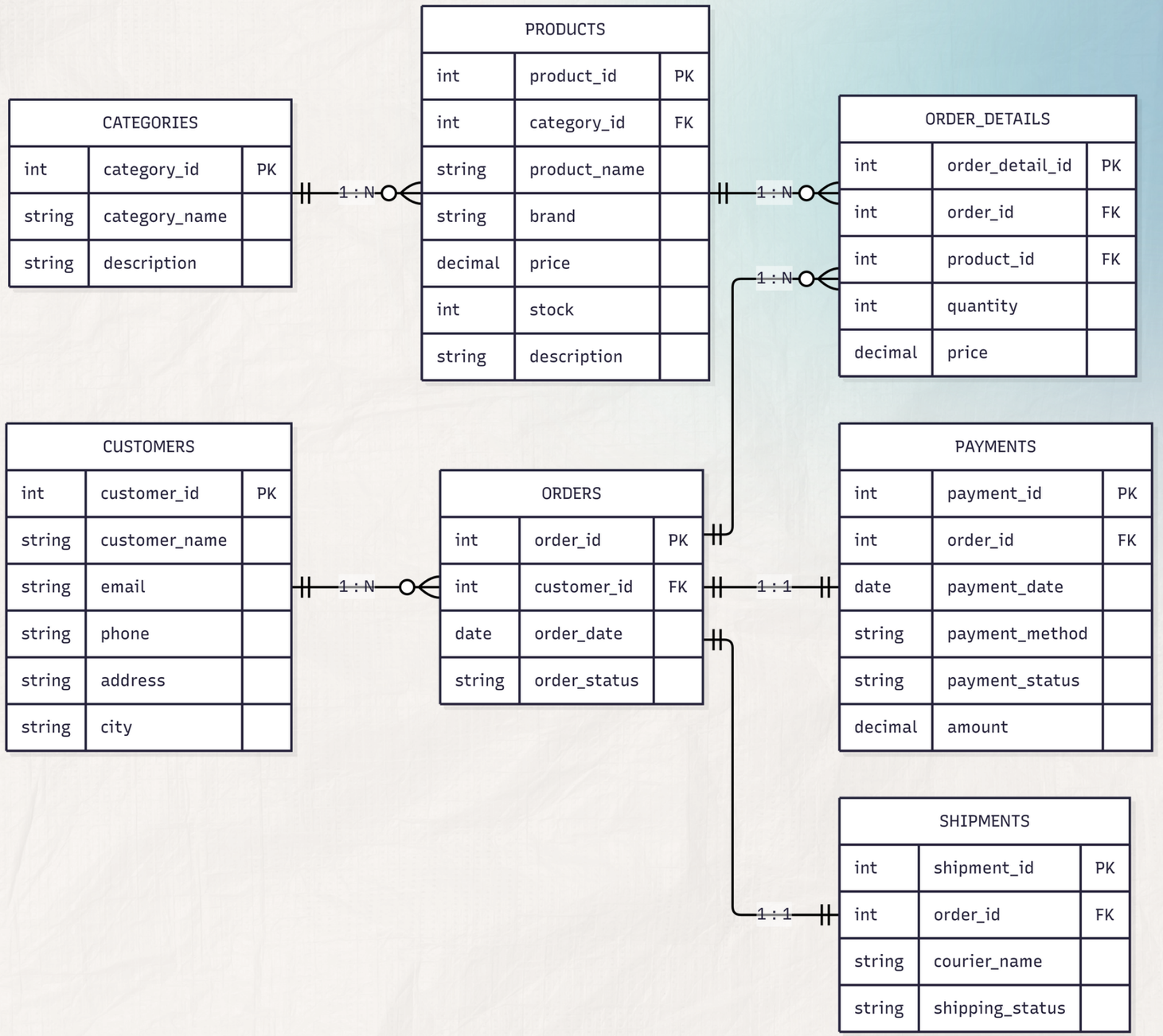
DATABASE STRUCTURE

No	Table Name	Description
1	CUSTOMERS	Data pelanggan
2	CATEGORIES	Kategori produk elektronik
3	PRODUCTS	Informasi produk, harga, dan stok
4	ORDERS	Data transaksi pemesanan
5	ORDER_DETAILS	Rincian produk dalam pesanan
6	PAYMENTS	Data pembayaran
7	SHIPMENTS	Data pengiriman

ENTITY RELATIONSHIP DIAGRAM (ERD)

Relationships & Cardinality

- CUSTOMERS – ORDERS (1:N)
- CATEGORIES – PRODUCTS (1:N)
- ORDERS – ORDER_DETAILS (1:N)
- PRODUCTS – ORDER_DETAILS (1:N)
- ORDERS – PAYMENTS (1:1)
- ORDERS – SHIPMENTS (1:1)



DATASET DESCRIPTION

Information	Description
Dataset Name	ElektroMart Dummy Dataset
Database	Oracle Database XE
SQL Tool	Oracle SQL Developer
Dataset Type	Synthetic / Dummy Data
Number of Tables	7 Tables

Table Name	Number of Records
CUSTOMERS	20
CATEGORIES	4
PRODUCTS	16
ORDERS	10
ORDER_DETAILS	20
PAYMENTS	10
SHIPMENTS	10
Total Record	**90**

- **Dataset dibuat secara dummy untuk simulasi operasional Elektromart dan bukan berasal dari transaksi pelanggan sungguhan.**

DATABASE IMPLEMENTATION

Mengimplementasikan perintah SQL CREATE TABLE untuk membangun 7 tabel yang saling terhubung menggunakan Primary Key (PK) dan Foreign Key (FK).

```sql
-- TABLE CUSTOMERS
CREATE TABLE CUSTOMERS (
    customer_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(100) NOT NULL,
    email VARCHAR2(100) NOT NULL,
    phone VARCHAR2(20) NOT NULL,
    address VARCHAR2(100) NOT NULL,
    city VARCHAR2(20) NOT NULL,
    created_at TIMESTAMP
);
```

Script Output | Task completed in 0.065 seconds

```
Table CUSTOMERS created.
```

```sql
-- TABLE ORDERS
CREATE TABLE ORDERS (
    order_id NUMBER PRIMARY KEY,
    customer_id NUMBER NOT NULL,
    order_date DATE NOT NULL,
    order_status VARCHAR2(30),
    total_amount NUMBER(12,2),
    CONSTRAINT fk_orders_customers
        FOREIGN KEY (customer_id)
        REFERENCES CUSTOMERS(customer_id)
);
```

Script Output | Task completed in 0.065 seconds

```
Table ORDERS created.
```

7 Tables Created

Script Output | Task completed in 0.065 seconds

```
Table CUSTOMERS created.

Table CATEGORIES created.

Table PRODUCTS created.

Table ORDERS created.

Table ORDER_DETAILS created.

Table PAYMENTS created.

Table SHIPMENTS created.
```

DATA INSERTION

Memasukkan data dummy ke dalam 7 tabel ElektroMart menggunakan perintah SQL INSERT INTO.

```
-- Data Dummy Products
INSERT ALL
    INTO PRODUCTS VALUES (1, 1, 'ASUS Vivobook', 'ASUS', 8500000, 15, 'Laptop untuk kerja dan kuliah')
    INTO PRODUCTS VALUES (2, 1, 'Lenovo IdeaPad', 'Lenovo', 7500000, 12, 'Laptop untuk kerja dan kuliah')
    INTO PRODUCTS VALUES (3, 1, 'Acer Aspire', 'Acer', 9200000, 10, 'Laptop untuk multitasking')
    INTO PRODUCTS VALUES (4, 2, 'Samsung Galaxy A55', 'Samsung', 5800000, 20, 'Smartphone untuk kebutuhan harian')
    INTO PRODUCTS VALUES (5, 2, 'iPhone 13', 'Apple', 10500000, 8, 'Smartphone Apple')
    INTO PRODUCTS VALUES (6, 2, 'Redmi Note 13', 'Xiaomi', 3200000, 25, 'Smartphone dengan harga terjangkau')
    INTO PRODUCTS VALUES (7, 3, 'Logitech Mouse', 'Logitech', 250000, 40, 'Mouse untuk komputer')
    INTO PRODUCTS VALUES (8, 3, 'Redragon Keyboard', 'Redragon', 450000, 30, 'Keyboard untuk komputer')
    INTO PRODUCTS VALUES (9, 3, 'Logitech Webcam', 'Logitech', 350000, 18, 'Webcam untuk meeting')
    INTO PRODUCTS VALUES (10, 4, 'JBL Headphone', 'JBL', 650000, 22, 'Headphone wireless')
    INTO PRODUCTS VALUES (11, 4, 'Sony Headphone', 'Sony', 780000, 16, 'Headphone bluetooth')
    INTO PRODUCTS VALUES (12, 4, 'JBL Speaker', 'JBL', 550000, 14, 'Speaker bluetooth')
    INTO PRODUCTS VALUES (13, 3, 'USB Flashdisk', 'SanDisk', 120000, 35, 'Media penyimpanan data')
    INTO PRODUCTS VALUES (14, 3, 'External Harddisk', 'Seagate', 950000, 10, 'Penyimpanan data eksternal')
    INTO PRODUCTS VALUES (15, 3, 'Laptop Stand', 'Baseus', 180000, 25, 'Dudukan laptop')
    INTO PRODUCTS VALUES (16, 4, 'JBL Earphone', 'JBL', 250000, 30, 'Earphone wireless')
SELECT 1 FROM DUAL;
COMMIT;
```

Script Output | Query Result | Query Result 1 | Query Result 2

Task completed in 0.073 seconds

```
16 rows inserted.
```

```
SELECT * FROM PRODUCTS;
```

Script Output | Query Result | Query Result 1 | Query Result 2

SQL | All Rows Fetched: 16 in 0.005 seconds

	PR...	CA...	PRODUCT_NAME	BRAND	PRICE	...	DESCRIPTION
1	1	1	ASUS Vivobook	ASUS	8500000	15	Laptop untuk...
2	2	1	Lenovo IdeaPad	Lenovo	7500000	12	Laptop untuk...
3	3	1	Acer Aspire	Acer	9200000	10	Laptop untuk...
4	4	2	Samsung Gal...	Samsung	5800000	20	Smartphone u...
5	5	2	iPhone 13	Apple	10500000	8	Smartphone A...
6	6	2	Redmi Note 13	Xiaomi	3200000	25	Smartphone d...
7	7	3	Logitech Mouse	Logitech	250000	40	Mouse untuk ...
8	8	3	Redragon Ke...	Redragon	450000	30	Keyboard unt...
9	9	3	Logitech We...	Logitech	350000	18	Webcam untuk...
10	10	4	JBL Headphone	JBL	650000	22	Headphone wi...
11	11	4	Sony Headphone	Sony	780000	16	Headphone bl...
12	12	4	JBL Speaker	JBL	550000	14	Speaker blue...
13	13	3	USB Flashdisk	SanDisk	120000	35	Media penyim...
14	14	3	External Ha...	Seagate	950000	10	Penyimpanan ...
15	15	3	Laptop Stand	Baseus	180000	25	Dudukan laptop
16	16	4	JBL Earphone	JBL	250000	30	Earphone wir...

DATA RETRIEVAL

Proses mengambil dan menampilkan data dari database sesuai kebutuhan informasi menggunakan perintah SQL.

1. Menampilkan produk dengan stok menipis

Tujuan: Mengidentifikasi produk dengan stok kurang dari 15 unit.

```sql
SELECT product_name, brand, stock
FROM PRODUCTS
WHERE stock < 15
ORDER BY stock ASC;
```

Script Output | Query Result 2

SQL | All Rows Fetched: 5 in 0.02 seconds

	PRODUCT_NAME	BRAND	STOCK
1	iPhone 13	Apple	8
2	Acer Aspire	Acer	10
3	External Harddisk	Seagate	10
4	Lenovo IdeaPad	Lenovo	12
5	JBL Speaker	JBL	14

2. Menampilkan pesanan yang sedang diproses

```sql
SELECT order_id, customer_id, order_date, order_status
FROM ORDERS
WHERE order_status = 'Processing'
ORDER BY order_date DESC;
```

Query Result

SQL | All Rows Fetched: 2 in 0.036 seconds

	ORDER_ID	CUSTOMER_ID	ORDER_DATE	ORDER_STATUS
1	9	17	09-10-2026	Processing
2	4	7	04-10-2026	Processing

3. Menampilkan informasi pembayaran yang belum lunas

```sql
SELECT
    payment_id,
    order_id,
    payment_method,
    payment_status,
    amount
FROM PAYMENTS
WHERE payment_status = 'Pending';
```

Script Output | Query Result

SQL | All Rows Fetched: 1 in 0.012 seconds

	PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS	AMOUNT
1	9	9	E-Wallet	Pending	1070000

4. Menampilkan 5 produk dengan harga tertinggi

```
SELECT product_name, brand, price
FROM PRODUCTS
ORDER BY price DESC
FETCH FIRST 5 ROWS ONLY;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0.003 seconds

	PRODUCT_NAME	BRAND	PRICE
1	iPhone 13	Apple	10500000
2	Acer Aspire	Acer	9200000
3	ASUS Vivobook	ASUS	8500000
4	Lenovo IdeaPad	Lenovo	7500000
5	Samsung Galaxy A55	Samsung	5800000

5. Menampilkan daftar kota pelanggan tanpa duplikasi

```
SELECT DISTINCT city
FROM CUSTOMERS
ORDER BY city ASC;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0.002 seconds

	CITY
1	Bekasi
2	Bogor
3	Depok
4	Jakarta
5	Tangerang

6. Menampilkan pelanggan yang namanya diawali huruf A.

```
SELECT customer_name, email, city
FROM CUSTOMERS
WHERE customer_name LIKE 'A%'
ORDER BY customer_name ASC;
```

Script Output | Query Result

SQL | All Rows Fetched: 6 in 0.002 seconds

	CUSTOMER_NAME	EMAIL	CITY
1	Althaf Ramadhan	althaf@gmail.com	Jakarta
2	Alya Rahmadani	alya@gmail.com	Depok
3	Arunika Putri	arunika@gmail.com	Bogor
4	Aurelia Nadhira	aurelia@gmail.com	Bekasi
5	Ayida Permatasari	ayida@gmail.com	Bogor
6	Ayu Lestari	ayu@gmail.com	Tangerang

7. Menampilkan produk dari beberapa merek tertentu

```
SELECT product_name, brand, price
FROM PRODUCTS
WHERE brand IN ('ASUS', 'Samsung', 'Apple')
ORDER BY brand ASC;
```

Script Output | Query Result

SQL | All Rows Fetched: 3 in 0.011 seconds

	PRODUCT_NAME	BRAND	PRICE
1	ASUS Vivobook	ASUS	8500000
2	iPhone 13	Apple	10500000
3	Samsung Galaxy A55	Samsung	5800000

DATA ANALYSIS

- Proses pengolahan dan penggabungan dari beberapa tabel untuk menghasilkan informasi bisnis ElektroMart.
- Menggunakan JOIN, GROUP BY, HAVING, serta fungsi agregasi seperti COUNT(), SUM(), dan AVG().

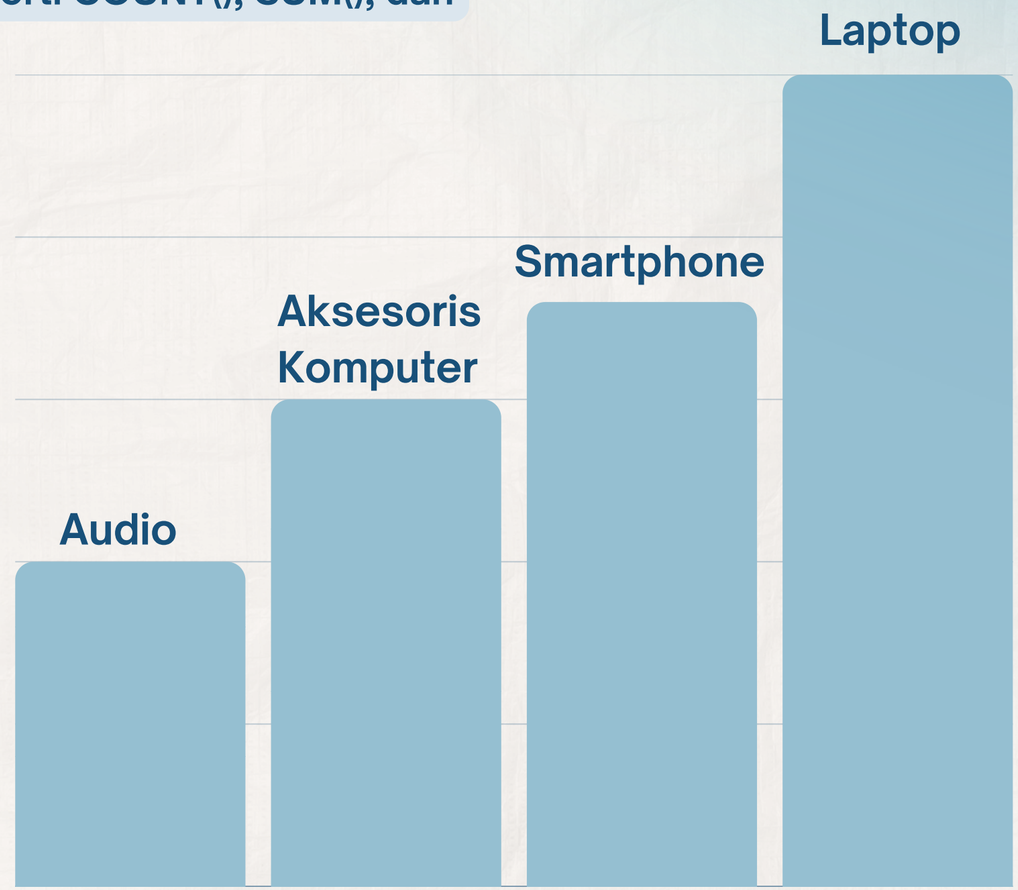
1. Menghitung total penjualan setiap kategori

```
SELECT
    c.category_name,
    SUM(od.quantity * od.price) AS total_penjualan
FROM CATEGORIES c
JOIN PRODUCTS p
    ON c.category_id = p.category_id
JOIN ORDER_DETAILS od
    ON p.product_id = od.product_id
GROUP BY c.category_name
ORDER BY total_penjualan DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0.02 seconds

	CATEGORY_NAME	TOTAL_PENJUALAN
1	Laptop	25200000
2	Smartphone	19500000
3	Aksesoris Komputer	3040000
4	Audio	2600000



**Untuk mengetahui kategori produk mana yang menghasilkan nilai penjualan tertinggi

2. **Menampilkan 5 produk dengan jumlah penjualan tertinggi.**

```
SELECT
    p.product_name,
    SUM(od.quantity) AS total_terjual
FROM PRODUCTS p
JOIN ORDER_DETAILS od
    ON p.product_id = od.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_terjual DESC
FETCH FIRST 5 ROWS ONLY;
```

Script Output | Query Result

SQL | All Rows Fetched: 5 in 0.024 seconds

	PRODUCT_NAME	TOTAL_TERJUAL
1	Logitech Mouse	3
2	JBL Earphone	3
3	USB Flashdisk	3
4	JBL Headphone	2
5	ASUS Vivobook	1

3. **Menghitung rata-rata harga produk setiap kategori**

```
SELECT
    c.category_name,
    ROUND(AVG(p.price), 2) AS harga_ratarata
FROM CATEGORIES c
JOIN PRODUCTS p
    ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY harga_ratarata DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0.015 seconds

	CATEGORY_NAME	HARGA_RATARATA
1	Laptop	8400000
2	Smartphone	6500000
3	Audio	557500
4	Aksesoris Komputer	383333,33

4. Menghitung jumlah pesanan berdasarkan status

```sql
SELECT
    order_status,
    COUNT(order_id) AS total_orders
FROM ORDERS
GROUP BY order_status
ORDER BY total_orders DESC;
```

Query Result

SQL | All Rows Fetched: 3 in 0.013 seconc

	ORDER_STATUS	TOTAL_ORDERS
1	Completed	6
2	Processing	2
3	Shipped	2

5. Mengetahui total pesanan setiap pelanggan

```sql
SELECT
    c.customer_name,
    COUNT(o.order_id) AS total_order
FROM CUSTOMERS c
JOIN ORDERS o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_order DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 10 in 0.041 seconds

	CUSTOMER_NAME	TOTAL_ORDER
1	Arunika Putri	1
2	Keisha Anindya	1
3	Nareswari Putri	1
4	Kaluna Maharani	1
5	Yustika Amalia	1
6	Saskia Larasati	1
7	Nayara Safitri	1
8	Tazkia Amara	1
9	Alya Rahmadani	1
10	Aurelia Nadhira	1

6. Menampilkan Produk dengan Jumlah Penjualan Lebih dari 1 Unit

```sql
SELECT
    p.product_name,
    SUM(od.quantity) AS total_terjual
FROM PRODUCTS p
JOIN ORDER_DETAILS od
    ON p.product_id = od.product_id
GROUP BY p.product_name
HAVING SUM(od.quantity) > 1
ORDER BY total_terjual DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0.007 seconds

	PRODUCT_NAME	TOTAL_TERJUAL
1	USB Flashdisk	3
2	Logitech Mouse	3
3	JBL Earphone	2
4	JBL Headphone	2

7. Menghitung Jumlah Pesanan Berdasarkan Status Pengiriman

```sql
SELECT
    shipping_status,
    COUNT(*) AS total_shipments
FROM SHIPMENTS
GROUP BY shipping_status
ORDER BY total_shipments DESC;
```

Script Output | Query Result

SQL | All Rows Fetched: 3 in 0.005 seconds

	SHIPPING_STATUS	TOTAL_SHIPMENTS
1	Delivered	7
2	Processing	1
3	Shipped	1

DATA MODIFICATION

Proses mengubah dan menghapus data dalam database sesuai kebutuhan.

1. Memperbarui stok produk

```
UPDATE PRODUCTS
SET stock = 25
WHERE product_id = 1;
```

Script Output | Query Result | Task completed in 0.075 seconds

```
1 row updated.
```

2. Memperbarui harga produk

```
UPDATE PRODUCTS
SET price = 8720000
WHERE product_name = 'ASUS Vivobook';
```

Script Output | Query Result | Task completed in 0.061 seconds

```
1 row updated.
```

3. Mengubah status pembayaran

```
UPDATE PAYMENTS
SET payment_status = 'Paid'
WHERE order_id = 4;
```

Script Output | Query Result | Task completed in 0.049 seconds

```
1 row updated.
```

4. Mengubah status pengiriman

```
UPDATE SHIPMENTS
SET shipping_status = 'Delivered'
WHERE order_id = 4;
```

Script Output | Query Result

Task completed in 0.056 seconds

```
1 row updated.
```

5. Memperbarui status pesanan

```
UPDATE ORDERS
SET order_status = 'Completed'
WHERE order_id = 4;
```

Script Output | Query Result

Task completed in 0.047 seconds

```
1 row updated.
```

6. Menambah stok produk

```
UPDATE PRODUCTS
SET stock = stock + 10
WHERE product_name = 'Laptop Stand';
```

Script Output | Query Result

Task completed in 0.05 seconds

```
1 row updated.
```

7. Mengganti deskripsi kategori

```
UPDATE CATEGORIES
SET description = 'Perangkat elektronik untuk kebutuhan sehari-hari'
WHERE category_name = 'Smartphone';
```

Script Output | Query Result

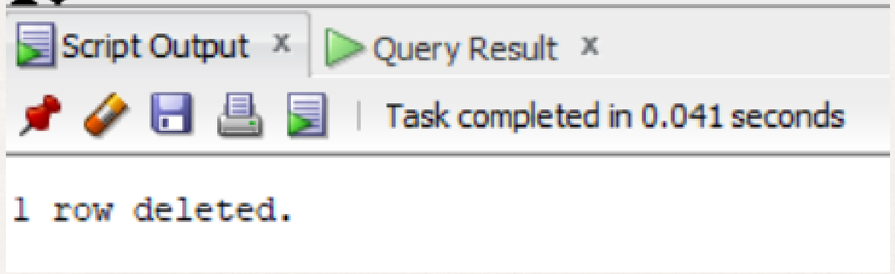
Task completed in 0.052 seconds

```
1 row updated.
```

Menghapus Transaksi Pesanan Beserta Relasinya pada ‘Order_id = 10’

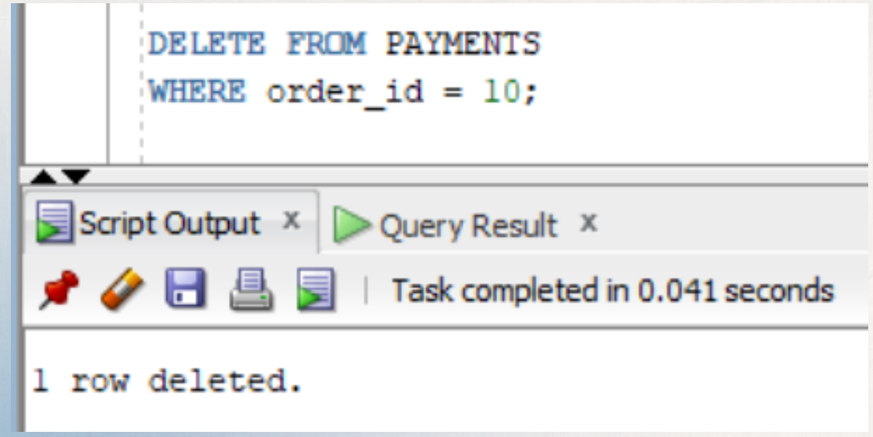
1. Menghapus ORDER_DETAIL

```
DELETE FROM ORDER_DETAILS
WHERE order_id = 10;
```



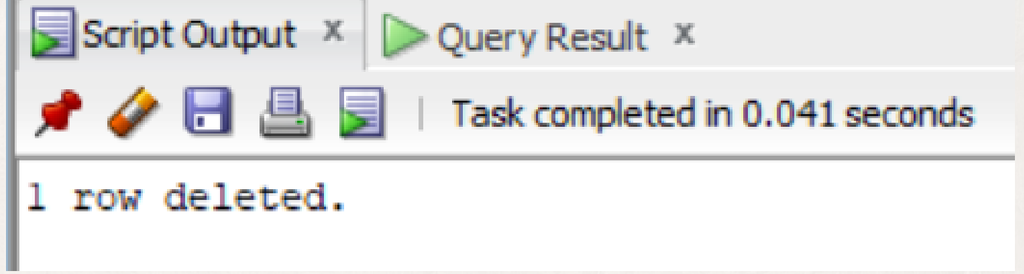
2. Menghapus PAYMENTS

```
DELETE FROM PAYMENTS
WHERE order_id = 10;
```



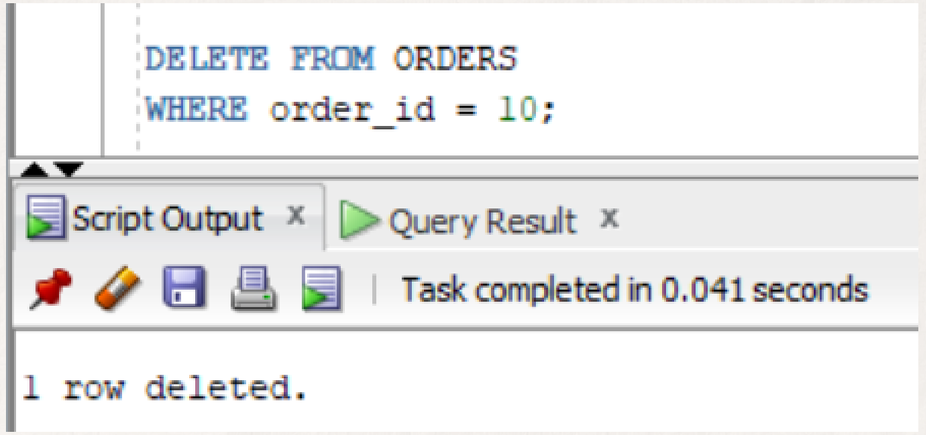
3. Menghapus SHIPMENTS

```
DELETE FROM SHIPMENTS
WHERE order_id = 10;
```



4. Menghapus ORDERS

```
DELETE FROM ORDERS
WHERE order_id = 10;
```



CONCLUSION

01

Database Design & Implementation

7 tabel relasional dengan PK dan FK.

02

SQL Implementation

Implementasi perintah SQL untuk pengelolaan data.

03

Data Analysis

Menghasilkan informasi terkait penjualan, pesanan, dan pengiriman.

04

Project Outcome

Pengelolaan data operasional secara terstruktur.

THANK YOU